

Lab: Infrastructure Automation and Hardening with Ansible

Master IoT

Dominique Dhoutaut

2025-2026

1 Lab Objectives

By the end of this lab, you will be able to :

1. Understand the Infrastructure as Code (IaC) philosophy and Ansible's model
 2. Install and configure Ansible on a control node
 3. Write Ansible playbooks to automate system administration tasks
 4. Use essential Ansible modules (apt, service, file, copy, template, lineinfile, ...)
 5. Manage variables, Jinja2 templates, and conditional logic
 6. Apply security best practices : SSH hardening, Ansible Vault, CIS Benchmarks
 7. Structure playbooks with reusable roles
 8. Orchestrate the deployment of multiple interdependent services
 9. Test and lint your playbooks with Ansible Lint
-

2 Preamble : Working Infrastructure

You have a Proxmox server on which you can create LXC containers. For this lab, you must prepare the following infrastructure :

- **1 LXC container “control-node”** : Ansible will be installed on this one. This will be your control machine.
- **3 LXC containers “managed-nodes”** : servers on which you will deploy and configure services via Ansible.

Name your containers consistently :

- **ansible-controller** (control node)
- **web-server** (managed node 1)
- **app-server** (managed node 2)
- **db-server** (managed node 3)

Each server will have the following characteristics :

- **root password** : “tpiot” on all containers

- **container type** : unprivileged
- **OS** : Debian 13 (using the template from previous sessions)
- **Storage** : 16GB disk
- **CPU** : 4 cores
- **RAM** : 1024MB
- **Network** : eth0 connected to the vmbrXXX bridge (the one with NAT allowing external connectivity)
- **Network** : IP addresses : 192.168.xxx.50, to 192.168.xxx.53

Start all 4 containers and verify that networking works :

- Open a terminal on each container
- Log in
- Verify that from one container you can ping the other 3
- Verify that the containers have internet access (if “ping www.google.com” is blocked by the university, use “**apt update**” instead)

Ensure that each container has an IP address accessible from the others, and that SSH access is enabled on all containers.

- SSH connections must be possible **between** the containers.
- It is not necessary to be able to SSH into the containers from the lab room.

3 Introduction to Ansible — Concepts and First Deployment

3.1 What is Infrastructure as Code (IaC) ?

Infrastructure as Code (IaC) is a practice of managing and provisioning infrastructure through text configuration files, rather than through manual procedures or graphical interfaces. The main advantages are :

- **Reproducibility** : infrastructure can be deployed identically each time
- **Traceability** : changes are versioned in a version control system (Git)
- **Collaboration** : multiple people can work on the same infrastructure
- **Audit** : you know exactly what was deployed and when
- **Speed** : deployment is automated and requires no manual intervention

3.2 Why Ansible ?

Among the available IaC tools (Terraform, Puppet, Chef, SaltStack, Ansible), Ansible stands out for :

- **Agentless** : no agent is needed on target machines. Ansible connects via SSH.
- **Push model** : the control node sends configurations to managed nodes.
- **YAML** : playbooks are written in YAML, a readable and simple format.
- **Modules** : Ansible has hundreds of modules to automate almost all common tasks.
- **Community** : an active community and many roles available on Ansible Galaxy.

3.3 Ansible Architecture

3.3.1 What is Ansible?

Ansible is an open-source infrastructure automation tool, developed by Red Hat. It allows you to configure systems, deploy applications, and orchestrate complex workflows across a large number of machines, in a declarative and reproducible manner.

How it works. Ansible is a command-line tool (CLI) that runs on a machine called the *control node*. It connects to target machines (*managed nodes*) via the SSH protocol (port 22 by default) to execute commands and scripts on them. There is no need to install an agent or daemon on the managed nodes : everything is done remotely via SSH. This makes Ansible *agentless* — unlike tools like Puppet or Chef that require an agent installed on each machine.

Technical implementation. Ansible is written in Python. Ansible modules (the small units of work that perform tasks such as installing a package, creating a file, starting a service) are also Python scripts. When Ansible executes a playbook, it transfers these Python scripts to the managed nodes via SSH, executes them, collects the results, and then deletes them. This is why managed nodes must have a Python interpreter (Python 3) — most modern Linux distributions include it by default. Ansible uses the `setup` module to automatically collect information about the managed nodes (IP address, operating system, memory, CPU, ...) : this information is called *facts* and can be used in playbooks to adapt behavior based on the target.

Declarative philosophy. With Ansible, you describe the *desired* state of the infrastructure (for example : “the nginx package must be installed, the service must be started and enabled at boot”), and Ansible figures out what needs to be done to reach that state. If the configuration is already compliant, Ansible does nothing (this is *idempotence*). This approach guarantees that repeated execution of a playbook always results in the same outcome, without side effects.

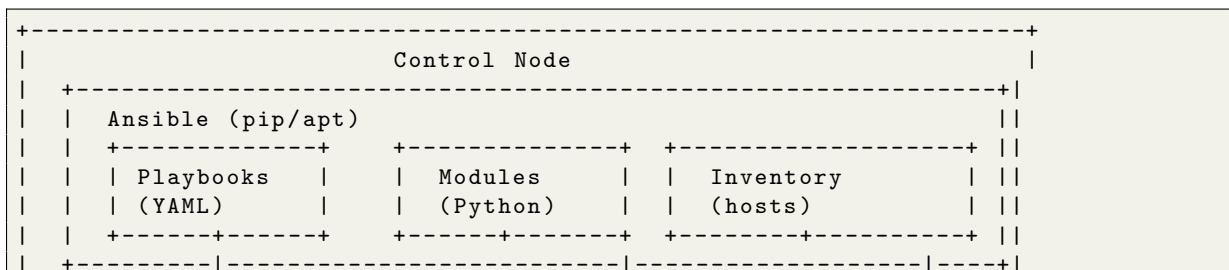
Description language. Ansible playbooks are written in YAML, a human-readable data format that is simple and widely used. A playbook is composed of one or more *plays*, each play associating a group of hosts with a series of *tasks*. Each task invokes an Ansible *module* with specific parameters.

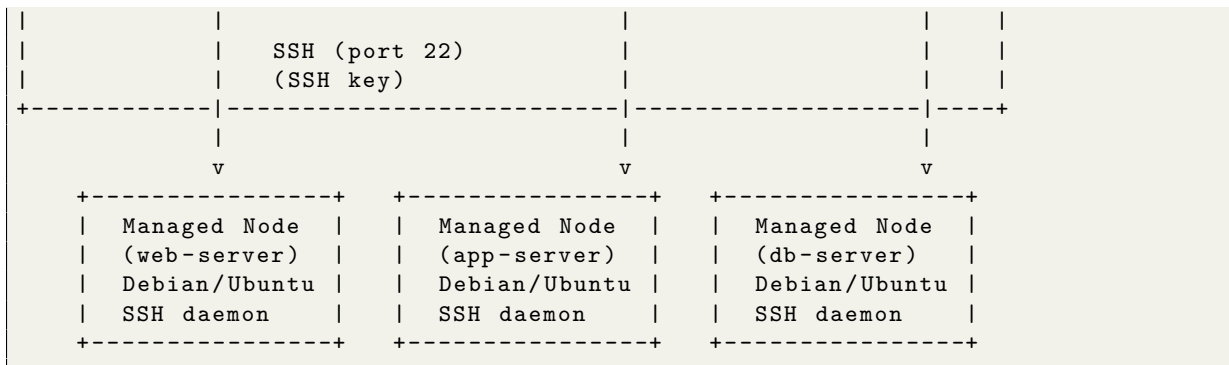
Three main components architecture.

- **Playbooks** : YAML files that describe the tasks to execute.
- **Modules** : Python scripts (transferred via SSH) that perform the actual work on managed nodes. Ansible provides hundreds of them (package management, services, files, users, networking, cloud, ...).
- **Inventory** : a file that lists target hosts and groups them, along with their connection variables (IP address, user, SSH key, ...).

Ad-hoc execution. In addition to playbooks, Ansible allows you to execute single commands directly from the command line, without writing a YAML file. These are called *ad-hoc commands*. For example, `ansible all -m ping` tests connectivity to all hosts in the inventory. This is useful for quick, one-off operations.

Here is a representation of Ansible’s architecture :





Excalidraw diagram : schemas/architecture-ansible.excalidraw

3.4 Installing Ansible

On your control node (`ansible-controller`), install Ansible :

Note that the commands below show “sudo” to illustrate that administrator rights are needed. But since you are logged in as “root”, you don’t need “sudo”.

```
# Update the system
sudo apt update && sudo apt upgrade -y

# Install Ansible
sudo apt install ansible -y

# Verify the installation
ansible --version
```

Listing 1 – Installing Ansible

3.5 SSH Configuration

For Ansible to connect to managed nodes, you must configure SSH. The recommended method is SSH key-based authentication.

Warning

Important for this lab : The `ssh-copy-id` command needs an initial authentication method to transfer the public key. Since your Proxmox container has root login disabled by default, the command will fail.

You must enable root login :

1. Edit `/etc/ssh/sshd_config`
2. Set `PermitRootLogin` to `yes`.
3. Save the file
4. Restart the `sshd` service : `systemctl restart sshd`
5. Verify that from “ansible-controller” you can connect as root to the other 3 containers (at this point you will need to enter the password).

Don’t worry : we will disable these insecure access methods later in the lab (Part 3) as part of hardening. If you get completely locked out, the Proxmox console is always accessible to restore the situation.

An alternative to this temporary change would be to generate the SSH key on “ansible-controller” and add it directly when creating the managed-nodes...

On the control node :

```
# Generate an SSH key pair (note: when ssh-keygen asks, use the default output
  file: /root/.ssh/id_ed25519)
ssh-keygen -t ed25519 -C "ansible@controller" -N ""

# Copy the public key to all managed nodes
ssh-copy-id root@<ip-web-server>
ssh-copy-id root@<ip-app-server>
ssh-copy-id root@<ip-db-server>
```

Listing 2 – SSH key generation and distribution

Test the connection (you should not need to enter a password) :

```
# from ansible-controller
ssh root@<ip-web-server> "hostname && uptime"
```

Listing 3 – Testing SSH connection

3.6 Host Inventory

Ansible's inventory is a file that lists the hosts Ansible can act upon. It can be static (text file) or dynamic (a script that generates the host list).

Create a static inventory file :

```
# ~/ansible/inventory.ini

[webservers]
web-server ansible_host=<ip-web-server>

[appservers]
app-server ansible_host=<ip-app-server>

[dbservers]
db-server ansible_host=<ip-db-server>

[all:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

Listing 4 – Static inventory file

Test connectivity with the ping module :

```
ansible all -i ~/ansible/inventory.ini -m ping
```

Listing 5 – Testing connectivity

You should get a SUCCESS response for each managed node.

Warning

Warning about the Python interpreter. You may see a warning like this :

```
[WARNING]: Host 'app-server' is using the discovered Python interpreter at
'/usr/bin/python3.13', but future installation of another Python
interpreter could cause a different interpreter to be discovered. See
https://docs.ansible.com/ansible-core/2.19/reference_appendices/
interpreter_discovery.html for more information.
```

Why this warning? Ansible needs Python to execute its modules on managed nodes. When it doesn't know which version of Python is installed, it *discovers* it automatically by running a small command on the target machine. This mechanism is convenient, but it carries a risk : if another Python interpreter is installed one day (e.g., via a playbook or a system update), Ansible might discover a different path and break. That's why it emits this warning to alert you.

How to resolve it? The best practice is to explicitly specify the Python interpreter path in the inventory, using the `ansible_python_interpreter` variable. Add this line to the `[all:vars]` section of your `inventory.ini` file :

```
[all:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_python_interpreter=/usr/bin/python3.13
```

Replace `/usr/bin/python3.13` with the actual path of Python 3 on your managed nodes (you can check with `ssh root@<ip> "which python3"`). Once this variable is set, Ansible will no longer need to discover the interpreter and the warning will disappear.

3.7 Ad-hoc Modules

Ad-hoc modules allow you to execute a single command on one or more hosts, without writing a playbook.

Understanding Ansible modules. Before moving to examples, it is essential to understand what an Ansible module is. A module is a reusable unit of code (usually a Python script) that performs a specific task on managed nodes. When you use `-m apt`, you are not calling the `apt` command from the Linux shell — you are invoking Ansible's *apt module*, which is a Python program designed to manage Debian packages in an idempotent way. This module will, internally, execute the real commands (`apt-get update`, `apt-get install`, ...), but it does so in a structured way : it first checks the current state, applies changes if necessary, and returns a clear result (changed, ok, failed).

Similarly, `-m service` is not the `systemctl` command : it is Ansible's `service` module, which knows how to start, stop, restart, or enable a service on different systems (systemd, sysvinit, ...). The `command` module executes a raw command without going through a shell, while `shell` goes through `/bin/sh` (which allows redirections and variables, but may present a risk).

Each module accepts specific *arguments* (passed via `-a`) that tell it exactly what to do. For example, the `apt` module accepts `name=nginx` (which package) and `state=present` (must be installed). This declarative approach — describing *what you want* rather than *how to do it* — is at the heart of Ansible's philosophy.

Ansible provides hundreds of modules covering almost all aspects of system administration : package management, services, files, users, databases, cloud (AWS, Azure, GCP), networking, containers, and more. The full list is available at docs.ansible.com.

Examples :

```
# Check disk space on all hosts
ansible all -i ~/ansible/inventory.ini -m command -a "df -h"

# Install a package on webserver
ansible webserver -i ~/ansible/inventory.ini -m apt -a "name=nginx state=
    present"

# Restart a service
ansible webserver -i ~/ansible/inventory.ini -m service -a "name=nginx state=
    restarted"

# Execute a shell command
ansible all -i ~/ansible/inventory.ini -m shell -a "uname -a"
```

Listing 6 – Ad-hoc module examples

3.8 First Playbook

An Ansible playbook is a YAML file that describes a series of tasks to execute on hosts.

Create your first playbook :

```
# ~/ansible/playbooks/01-system-update.yml
---
- name: First playbook - System update
  hosts: all
  become: true

  tasks:
    - name: Update packages
      apt:
        update_cache: true
        upgrade: dist

    - name: Install common packages
      apt:
        name:
          - curl
          - wget
          - vim
          - htop
        state: present

    - name: Check if a reboot is required
      stat:
        path: /var/run/reboot-required
        register: reboot_required

    - name: Reboot the machine if necessary
      reboot:
        connect_timeout: 5
        when: reboot_required.stat.exists
```

Listing 7 – First playbook: system update

Execute the playbook :

```
ansible-playbook ~/ansible/playbooks/01-system-update.yml -i ~/ansible/inventory
.ini
```

Listing 8 – Executing the playbook

You should see your servers being updated, and eventually rebooting. Note that this playbook is relatively simple (its structure will be detailed in the next section).

4 Fundamentals — Playbooks, Modules, and Variables

4.1 Playbook Structure — Detailed Analysis

Let's revisit the playbook from section 1.8 and analyze each line :

```
# ~/ansible/playbooks/01-system-update.yml
---
- name: First playbook - System update
  hosts: all
  become: true

  tasks:
    - name: Update packages
      apt:
        update_cache: true
        upgrade: dist

    - name: Install common packages
      apt:
        name:
          - curl
          - wget
          - vim
          - htop
        state: present

    - name: Check if a reboot is required
      stat:
        path: /var/run/reboot-required
      register: reboot_required

    - name: Reboot the machine if necessary
      reboot:
        connect_timeout: 5
      when: reboot_required.stat.exists
```

Listing 9 – First playbook: system update

Line-by-line analysis :

- `--` : Start of the YAML document. This separator indicates the beginning of a new YAML document.
- `- name: First playbook - System update` : Start of a *play*. The `name` is a descriptive label that will appear in Ansible's output during execution. It does not affect behavior but aids readability.
- `hosts: all` : Specifies that this play applies to *all* hosts defined in the inventory. You could also target a specific group like `webserver`s or `dbserver`s.
- `become: true` : Indicates that Ansible should use `sudo` (or equivalent) to execute tasks. This is necessary because package updates and system reboots require root privileges.
- `tasks:` : Start of the task list. Each task is an individual action that Ansible will execute.
- `- name: Update packages` : Name of the first task. Descriptive and clear.
- `apt:` : Invokes Ansible's *apt module* (not the shell's `apt` command). This module manages Debian packages in an idempotent way.

- `update_cache: yes` : Argument for the `apt` module. Tells Ansible to first run `apt-get update` to refresh the list of available packages.
- `upgrade: dist` : Argument for the `apt` module. Tells Ansible to perform a full system upgrade (`apt-get dist-upgrade`), not just the installed packages.
- `- name: Install common packages` : Name of the second task.
- `apt:` : We invoke the `apt` module again.
- `name:` : List of packages to install. In YAML, `-` indicates a list item.
- `state: present` : Argument for the `apt` module. Indicates that packages must be *present* (installed). If a package is already installed, Ansible does nothing (idempotence).
- `- name: Check if a reboot is required` : Name of the third task.
- `stat:` : Invokes Ansible's *stat module*. This module checks for the existence of a file on the managed node and returns information about it (existence, permissions, etc.).
- `path: /var/run/reboot-required` : Argument for the `stat` module. On Debian/Ubuntu systems, this file is automatically created after a Linux kernel update if a reboot is required.
- `register: reboot_required` : The `register` statement stores the result of the `stat` module in a variable named `reboot_required`. This variable can be used in subsequent tasks via `reboot_required.stat.exists` (which will be `true` or `false`).
- `- name: Reboot the machine if necessary` : Name of the fourth task.
- `reboot:` : Invokes Ansible's *reboot module*. This module safely reboots the machine while waiting for the SSH service to become available again.
- `connect_timeout: 5` : Argument for the `reboot` module. Specifies the wait time in seconds before considering the reboot as failed.
- `when: reboot_required.stat.exists` : *when* condition. This task only executes if the `/var/run/reboot-required` file exists, i.e., only if a reboot is actually required after the kernel update. Unlike a condition based on `ansible_os_family`, this approach is more precise : it only reboots if the system actually requests it.

Summary of what this playbook does :

1. Updates the list of available packages and installs the latest versions
2. Installs common utility packages (`curl`, `wget`, `vim`, `htop`)
3. Checks if a reboot is required (the `/var/run/reboot-required` file)
4. Reboots the machine only if a reboot is required

This playbook is *idempotent* : if you run it multiple times, it will only apply changes if they are necessary. For example, if all packages are already up to date, the first task will do nothing.

4.2 Second Example : Web Server Deployment

Here is a more complete playbook that deploys an Nginx web server. This time, the playbook is part of a larger project that also includes a template for an Nginx vhost configuration, as well as a basic PHP file to deploy :

```
02-nginx-php-fpm/
+-- playbook.yaml
+-- templates/
|   +-- nginx-site.conf.j2
+-- files/
    +-- index.php
```

Some reminders before we begin :

PHP-FPM (FastCGI Process Manager) is a process manager for PHP. Nginx cannot execute PHP on its own : it delegates the processing of `.php` files to PHP-FPM via a Unix socket (here

/run/php/php8.4-fpm.sock). PHP-FPM runs in the background, receives requests from Nginx, processes them, and returns the HTML result.

Multi-site management in Nginx : Nginx uses a symbolic link mechanism to easily enable or disable sites. Configuration files for each site are placed in `/etc/nginx/sites-available/` (sites *available*). To enable a site, you create a symbolic link in `/etc/nginx/sites-enabled/` pointing to the corresponding file. Nginx automatically includes all files from `sites-enabled/` in its main configuration. This allows you to have as many sites as you want without modifying Nginx's configuration.

The playbook.yaml file :

```
---
- name: Nginx + PHP 8.4 Deployment (default site)
  hosts: webserver
  become: true

  vars:
    nginx_port: 80
    web_root: /var/www/html

    php_version: "8.4"
    php_fpm_service: "php{{ php_version }}-fpm"
    php_fpm_socket: "/run/php/php{{ php_version }}-fpm.sock"

    server_name: _

  tasks:

    - name: Update apt cache
      apt:
        update_cache: yes

    - name: Install Nginx and PHP 8.4
      apt:
        name:
          - nginx
          - "php{{ php_version }}-fpm"
          - php-cli
        state: present

    - name: Create web directory
      file:
        path: "{{ web_root }}"
        state: directory
        owner: www-data
        group: www-data
        mode: "0755"

    - name: Deploy PHP info page
      copy:
        src: files/index.php
        dest: "{{ web_root }}/index.php"
        owner: www-data
        group: www-data
        mode: "0644"

    - name: Deploy Nginx configuration
      template:
        src: templates/nginx-site.conf.j2
        dest: /etc/nginx/sites-available/default
        mode: "0644"
      notify: Reload nginx
```

```

- name: Enable the default site
  file:
    src: /etc/nginx/sites-available/default
    dest: /etc/nginx/sites-enabled/default
    state: link
    force: true

- name: Start and enable services
  systemd:
    name: "{{ item }}"
    state: started
    enabled: true
  loop:
    - nginx
    - "{{ php_fpm_service }}"

handlers:
- name: Reload nginx
  systemd:
    name: nginx
    state: reloaded

```

Listing 10 – Nginx + PHP 8.4 deployment playbook

The nginx-site.conf.j2 file :

```

server {
    listen {{ nginx_port }};
    server_name {{ server_name }};

    root {{ web_root }};
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:{{ php_fpm_socket }};
    }
}

```

Listing 11 – Nginx site configuration template

The index.php file :

```

<?php
phpinfo();
?>

```

Listing 12 – PHP info page

Analysis of new features compared to the first playbook :

- **vars:** : Variable definition section. `nginx_port`, `web_root`, `php_version`, `php_fpm_service`, `php_fpm_socket`, and `server_name` are playbook-local variables. They can be used in tasks and templates via the `{{ variable }}` syntax.
- **file:** : The *file module* manages files and directories. Here, it is used to create the web root directory (`state: directory`) with `www-data` ownership and `0755` permissions.

- **copy:** : The *copy module* copies a file from the control node to managed nodes. The **src**, **dest**, **owner**, **group**, **mode** arguments control the source, destination, owner, group, and permissions of the copied file, respectively.
- **template:** : The *template module* works like **copy**, but interprets Jinja2 syntax (`{{ variable }}`) in the source file before copying. This allows generating dynamic configuration files based on playbook variables.
- **notify:** Reload **nginx** : This instruction *notifies* a handler named “Reload nginx”. The handler will only execute at the end of the play, and only if the notifying task actually changed something (i.e., if the template was regenerated).
- **file:** with **state:** **link** : This task creates a symbolic link in `/etc/nginx/sites-enabled/` pointing to the configuration enabled in `sites-available`. The **force: true** argument allows overwriting the link if it already exists.
- **systemd:** : The *systemd module* manages system services (it replaces the **service** module in recent versions of Ansible). **state: started** starts the service, **enabled: true** enables it at system boot.
- **loop:** : This instruction iterates over a list of items. Here, it applies the same **systemd** task to both **nginx** and the PHP-FPM service, avoiding task duplication.
- **handlers:** : Section defining handlers. A handler is a special task that only executes when notified. Here, the “Reload nginx” handler uses the **systemd** module with **state: reloaded** to reload Nginx’s configuration without interrupting active connections (unlike a full restart).

Execution flow :

1. Update apt cache
2. Install Nginx and PHP 8.4
3. Create the web root directory
4. Deploy the test PHP page
5. Deploy Nginx configuration via a Jinja2 template
6. Enable the site via symbolic link
7. Start and enable Nginx and PHP-FPM services
8. Reload Nginx (only if the configuration changed)

4.3 Essential Modules

Here are the most commonly used modules :

4.4 Variables and Facts

Ansible has several types of variables :

1. **Inline variables** : defined directly in the playbook
2. **Inventory variables** : defined in the inventory file
3. **Variables in separate files** : defined in `vars/` files
4. **Facts** : variables automatically discovered by Ansible (OS, IP, memory, ...)

Example of variables in a playbook :

```
---
- name: Deployment with variables
  hosts: all
  become: true
  vars:
```

Module	Description	Example
apt	Debian package management	apt: name=nginx state=present
service	Service management	service: name=nginx state=started enabled=yes
file	File and directory management	file: path=/tmp/test state=directory mode='0755'
copy	File copying	copy: src=local dest=/remote owner=root mode='0644'
template	File copying with Jinja2 templating	template: src=template.j2 dest=/remote
lineinfile	Line-by-line file modification	lineinfile: path=/etc/ssh/sshd_config line="PermitRootLogin no"
user	User management	user: name=alice groups=sudo shell=/bin/bash
command	Command execution	command: /usr/bin/foo
shell	Command execution via shell	shell: echo \$HOME
yum	RPM package management	yum: name=nginx state=present
firewalld	firewalld firewall management	firewalld: service=ssh permanent=true state=enabled
ufw	UFW firewall management	ufw: rule=allow port=22 proto=tcp

TABLE 1 – Essential Ansible modules

```

common_packages:
  - curl
  - wget
  - vim
ntp_server: "pool.ntp.org"
tasks:
  - name: Install common packages
    apt:
      name: "{{ common_packages }}"
      state: present

  - name: Display facts
    debug:
      msg: "OS: {{ ansible_distribution }}, IP: {{ ansible_default_ipv4.
address }}"

```

Listing 13 – Variables example

To see all available facts, use the “setup” module :

```
ansible all -i ~/ansible/inventory.ini -m setup
```

Listing 14 – Display all facts

But you will notice that the number of collected facts is considerable, and you may want to filter.

For example, to display only machine names as well as the OS type and version :

```
ansible all -i ~/ansible/inventory.ini -m setup -a "filter=ansible_hostname,
ansible_distribution,ansible_distribution_version"
```

Listing 15 – Filter facts

Or to display only network-related facts :

```
ansible all -i ~/ansible/inventory.ini -m setup -a "gather_subset=network"
```

Listing 16 – Network facts only

4.5 Handlers and Notifications

Handlers are tasks that only execute when notified by another task. They are useful for restarting services after a configuration change.

```
---
- name: Nginx Configuration
  hosts: webserver
  become: true
  tasks:
    - name: Copy Nginx configuration
      copy:
        src: files/nginx.conf
        dest: /etc/nginx/nginx.conf
      notify: Restart Nginx

    - name: Copy welcome file
      copy:
        content: "Welcome to our server!"
        dest: /var/www/html/index.html
      notify:
        - Restart Nginx
        - Check Nginx configuration

  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted

    - name: Check Nginx configuration
      command: nginx -t
```

Listing 17 – Handlers example

Important : A handler is executed only once, even if it is notified multiple times.

5 Security — SSH Keys, Ansible Vault, and Hardening

5.1 The Importance of Security in Automation

Infrastructure automation raises significant security concerns :

- **SSH access :** SSH keys must be protected and rotated regularly
- **Secrets :** database passwords, API keys, TLS certificates must be encrypted
- **Hardening :** servers must be configured according to security benchmarks (CIS, STIG)

- **Privileges** : the principle of least privilege must be applied
- **Audit** : all changes must be traceable

5.2 Secure SSH Configuration

5.2.1 SSH Keys

Ansible uses SSH to connect to managed nodes. The security of this connection is paramount.

Best practices :

- Use Ed25519 keys (more secure than RSA)
- Do not use a passphrase for automation (but protect the key file)
- Restrict SSH access with `AllowUsers` or `AllowGroups`
- Disable password authentication

5.2.2 SSH Hardening with Ansible

Context and authentication strategy. So far, we have connected to managed nodes as `root` to run our playbooks. This practice is acceptable during testing, but unthinkable in production. Allowing `root` connections, even with an SSH key, exposes the system to major risks : any key compromise gives immediate full access, and logs cannot distinguish whether an action was performed by Ansible, an administrator, or a system process.

To address this, we adopt a two-step approach :

1. **Create a dedicated administration user (`ansible`).** This user will have `sudo` rights to execute system administration tasks. SSH connections will then be restricted to this user only. This allows better traceability : in the logs, you will know exactly that the action comes from the automation process. Additionally, this user's `sudo` rights can be refined later to allow only a subset of commands (principle of least privilege).
2. **Create a dedicated deployment user (`deploy`).** This user is designed for automating application deployments via CI/CD pipelines (GitHub Actions, Jenkins, GitLab CI, etc.). In this example, we will not actually create the `deploy` account, but we will mention it in the SSH configuration to show how to structure access in production. In a real situation, this user would have even more restricted `sudo` rights (or none at all) : it could only execute commands strictly necessary for deployments (file copying, reloading specific services, etc.).

This separation of roles (`ansible` for system administration, `deploy` for application deployments) is a fundamental DevOps best practice. It limits the attack surface, ensures clear traceability of actions, and prepares the ground for fine-grained access governance via `sudoers`.

Create a playbook to harden the SSH configuration :

```
# ~/ansible/playbooks/03-ansible-user.yml
---
- name: Bootstrap ansible user + SSH key
  hosts: all
  become: true

  vars:
    ansible_user_name: ansible
    ansible_ssh_pubkey_path: "~/.ssh/id_ed25519.pub"

  tasks:

    - name: Install sudo (Debian)
      apt:
```

```

    name: sudo
    state: present
    update_cache: true

- name: Create the ansible user
  user:
    name: "{{ ansible_user_name }}"
    shell: /bin/bash
    create_home: true

- name: Create the .ssh directory
  file:
    path: "/home/{{ ansible_user_name }}/.ssh"
    state: directory
    owner: "{{ ansible_user_name }}"
    group: "{{ ansible_user_name }}"
    mode: "0700"

- name: Deploy the control node's public SSH key
  authorized_key:
    user: "{{ ansible_user_name }}"
    state: present
    key: "{{ lookup('file', ansible_ssh_pubkey_path) }}"

- name: Grant sudo rights to ansible (NOPASSWD)
  copy:
    dest: "/etc/sudoers.d/{{ ansible_user_name }}"
    content: "{{ ansible_user_name }} ALL=(ALL) NOPASSWD:ALL\n"
    mode: "0440"
    validate: "visudo -cf %s"

- name: Verify that sudo works without a password
  command: sudo -n true
  become: true
  become_user: "{{ ansible_user_name }}"
  changed_when: false
  failed_when: false

```

Listing 18 – Bootstrap ansible user + SSH key

```

# ~/ansible/playbooks/03-ssh-hardening.yaml
---
- name: SSH Hardening (simplified)
  hosts: all
  become: true

  vars:
    ssh_port: 22
    ssh_allowed_users: "ansible deploy"

  tasks:

    - name: Install OpenSSH server
      package:
        name: openssh-server
        state: present

    - name: Configure SSH - port
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?Port'
        line: "Port {{ ssh_port }}"
        validate: '/usr/sbin/sshd -t -f %s'

```



```

- name: Disable root login
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: 'PermitRootLogin no'
    validate: '/usr/sbin/sshd -t -f %s'

- name: Disable password authentication
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PasswordAuthentication'
    line: 'PasswordAuthentication no'
    validate: '/usr/sbin/sshd -t -f %s'

- name: Allow only specific users
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?AllowUsers'
    line: "AllowUsers {{ ssh_allowed_users }}"
    validate: '/usr/sbin/sshd -t -f %s'

- name: Enable verbose logging
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?LogLevel'
    line: 'LogLevel VERBOSE'
    validate: '/usr/sbin/sshd -t -f %s'

- name: Restart SSH (Debian/RHEL compatible)
  service:
    name: "{{ 'ssh' if ansible_os_family == 'Debian' else 'sshd' }}"
    state: restarted

```

Listing 19 – SSH Hardening (simplified)

Warning

Before changing the SSH port or disabling root login, make sure you have another way to access the server (e.g., Proxmox console).

5.3 Ansible Vault

Why encrypt secrets? In any automation project, some files contain sensitive information : database passwords, API keys, SSL certificates, authentication tokens, etc. This data must never be stored in plaintext, especially if the code is versioned in a Git repository (even a private one). A plaintext secrets file in a repository immediately exposes this information to anyone with access to the repository, and makes it difficult to revoke in case of compromise.

The principle of Ansible Vault. Ansible Vault solves this problem by encrypting complete YAML files or individual variables. The encryption uses AES-256, a robust standard widely used. A file encrypted by Ansible Vault looks like a normal YAML file, but starts with `$ANSIBLE_VAULT;1.1;AES256` followed by base64-encoded data. Without the correct password, this data is unreadable.

How it works in practice. When a playbook references an encrypted file (via `vars_files` or `include_vars`), Ansible asks for the password to decrypt it in memory before using it. The file remains encrypted on disk — only the in-memory content is decrypted. This means :

- You can commit encrypted files to Git without risk

- The password must never be stored in code or in the secrets file itself
- Each playbook execution requires providing the password (or a password file)

Password management. There are several ways to provide the password to Ansible :

- **Interactive mode** (`-ask-vault-pass`) : Ansible prompts for the password during execution. Simple for testing, but impractical for automation.
- **Password file** (`-vault-password-file`) : Ansible reads the password from a file. Useful for scripts, but the file must be protected (permissions 600) and must not be versioned.
- **Environment variables** (`ANSIBLE_VAULT_PASSWORD_FILE`) : Allows configuring behavior without modifying the command.
- **CI/CD integration** : In a pipeline (GitHub Actions, Jenkins, GitLab CI), the password is stored in the pipeline's secure environment variables and passed to Ansible Vault automatically.

Best practices.

- Never store the Vault password in code or in a versioned file
- Use a dedicated secrets manager (HashiCorp Vault, AWS Secrets Manager, etc.) in production
- Encrypt only the files that need it (not the entire playbook)
- Use separate secrets files per environment (development, staging, production)
- Regularly rotate Vault passwords and the secrets they contain

What we will do. In the following sections, we will :

1. Create an encrypted secrets file containing database passwords and API keys
2. Structure this file so it is easy to maintain and use
3. Integrate these secrets into a playbook that configures a MySQL database

5.3.1 Creating an Encrypted File

```
# Create an encrypted file
ansible-vault create ~/ansible/secrets.yml

# Encrypt an existing file
ansible-vault encrypt ~/ansible/secrets.yml

# Decrypt and display
ansible-vault view ~/ansible/secrets.yml

# Edit an encrypted file
ansible-vault edit ~/ansible/secrets.yml
```

Listing 20 – Ansible Vault commands

5.3.2 Contents of a Secrets File

```
# ~/ansible/secrets.yml (encrypted)
---
db_root_password: "S3cur3P@ssw0rd!"
db_app_password: "AppP@ssw0rd!"
api_key: "sk-123...cdef"
ssl_certificate: |
  -----BEGIN CERTIFICATE-----
  MIID...
  -----END CERTIFICATE-----
```

Listing 21 – Encrypted secrets file

5.3.3 Using Secrets in a Playbook

```
# ~/ansible/playbooks/03-use-vault.yml
---
- name: Using Ansible Vault
  hosts: dbbservers
  become: true
  vars_files:
    - ../secrets.yml
  tasks:
    - name: Display the password (for demonstration)
      debug:
        msg: "The DB password is: {{ db_root_password }}"

    - name: Create the database user
      mysql_user:
        name: app_user
        password: "{{ db_app_password }}"
        priv: "app_db.*:ALL"
        state: present
```

Listing 22 – Using Ansible Vault in a playbook

Execution with the vault password :

```
ansible-playbook ~/ansible/playbooks/03-use-vault.yml -i ~/ansible/inventory.ini
--ask-vault-pass
```

Listing 23 – Executing with vault password

Or with a password file :

```
echo "my_password" > ~/ansible/vault_password.txt
chmod 600 ~/ansible/vault_password.txt
ansible-playbook ~/ansible/playbooks/03-use-vault.yml -i ~/ansible/inventory.ini
--vault-password-file ~/ansible/vault_password.txt
```

Listing 24 – Executing with password file

5.4 User and Permission Management

5.4.1 Creating Users with Specific Roles

```
# ~/ansible/playbooks/03-user-management.yml
---
- name: User management
  hosts: all
  become: true
  tasks:
    - name: Create the deploy group
      group:
        name: deploy
        state: present

    - name: Create the deploy user
      user:
        name: deploy
        groups: deploy
        shell: /bin/bash
        create_home: yes
        ssh_authorized_keys:
          - "ssh-ed25519 AAAA... ansible@controller"
```

```

- name: Configure sudo for the deploy group
  lineinfile:
    path: /etc/sudoers.d/deploy
    line: '%deploy ALL=(ALL) NOPASSWD: ALL'
    validate: 'visudo -cf %s'
    create: yes
    mode: '0440'

```

Listing 25 – User management playbook

5.4.2 File Permissions

```

- name: Set permissions on a sensitive file
  file:
    path: /etc/myapp/secrets.conf
    owner: root
    group: root
    mode: '0600'

- name: Create a directory with specific permissions
  file:
    path: /var/log/myapp
    owner: myapp
    group: myapp
    mode: '0750'
    state: directory

```

Listing 26 – File permissions management

5.5 Introduction to CIS Hardening

CIS Benchmarks are security recommendations for hardening operating systems. For Debian/Ubuntu, here are the key points :

1. **SSH** : disable root login, password auth, use SSH keys
2. **Passwords** : complexity policy, expiration, history
3. **Firewall** : enable and configure UFW or firewalld
4. **Logging** : enable and configure rsyslog/journald
5. **Updates** : enable automatic security updates
6. **Services** : disable unnecessary services
7. **Permissions** : apply the principle of least privilege

6 Jinja2 Templates and Conditional Logic

6.1 Introduction to Jinja2

Jinja2 is a Python template engine, integrated into Ansible. It allows generating dynamic configuration files based on variables.

Unlike the `copy` module which copies a file as-is, the `template` module evaluates Jinja2 variables before copying the file.

6.2 Basic Syntax

```
# Simple variables
Hello {{ name }}, you are {{ age }} years old.

# Conditions
{% if age >= 18 %}
    You are an adult.
{% else %}
    You are a minor.
{% endif %}

# Loops
<ul>
{% for service in services %}
    <li>{{ service.name }} (port {{ service.port }})</li>
{% endfor %}
</ul>

# Filters
{{ name | upper }}
{{ liste | join(', ') }}
{{ valeur | default('default value') }}
```

Listing 27 – Jinja2 basic syntax

6.3 Example : Nginx Template

Create an Nginx template :

```
# ~/ansible/templates/nginx.conf.j2
server {
    listen {{ nginx_port | default(80) }};
    server_name {{ nginx_server_name | default('localhost') }};

    root {{ nginx_root | default('/var/www/html') }};
    index index.html index.htm;

{% if ssl_enabled | default(false) %}
    listen 443 ssl;
    ssl_certificate {{ ssl_cert_path | default('/etc/ssl/certs/server.crt') }};
    ssl_certificate_key {{ ssl_key_path | default('/etc/ssl/private/server.key') }};
{% endif %}

    location / {
        try_files $uri $uri/ =404;
    }

{% if enable_gzip | default(true) %}
    gzip on;
    gzip_types text/plain text/css application/json application/javascript;
{% endif %}
}
```

Listing 28 – Nginx Jinja2 template

Usage in a playbook :

```
---
- name: Configure Nginx with template
  hosts: webservers
  become: true
```

```

vars:
  nginx_port: 80
  nginx_server_name: "monsite.example.com"
  nginx_root: "/var/www/monsite"
  ssl_enabled: false
  enable_gzip: true
tasks:
  - name: Create root directory
    file:
      path: "{{ nginx_root }}"
      state: directory
      owner: www-data
      group: www-data
      mode: '0755'

  - name: Copy Nginx template
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/sites-available/monsite
      owner: root
      group: root
      mode: '0644'
    notify: Reload Nginx

  - name: Enable the site
    file:
      src: /etc/nginx/sites-available/monsite
      dest: /etc/nginx/sites-enabled/monsite
      state: link
    notify: Reload Nginx

```

Listing 29 – Nginx template usage in playbook

6.4 Conditions and Logic

6.4.1 Conditions with when

```

- name: Install Apache only on CentOS
  apt:
    name: apache2
    state: present
    when: ansible_os_family == "Debian"

- name: Configure firewall only if UFW is installed
  ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  with_items:
    - "80"
    - "443"
  when: ansible_facts['pkg_mgr'] == "apt"

```

Listing 30 – Conditional execution with when

6.4.2 Loops

```

- name: Install multiple packages
  apt:
    name: "{{ item }}"
    state: present
  loop:

```

```

- curl
- wget
- vim
- htop

- name: Configure ports to allow
  ufw:
    rule: allow
    port: "{{ item.key }}"
    proto: "{{ item.value }}"
    loop: "{{ ports_to_allow | dict2items }}"

```

Listing 31 – Loop examples

7 Ansible Roles and Reusability

7.1 Why Roles ?

Roles allow you to structure and reuse playbooks. Instead of having a single YAML file containing all tasks, a role organizes code into logical subdirectories.

Advantages :

- **Modularity** : each role does one thing
- **Reusability** : a role can be used in multiple playbooks
- **Sharing** : roles can be shared with the community via Ansible Galaxy
- **Organization** : code is better structured and easier to maintain

Example : **Without a role** :

```

- hosts: all
  tasks:
    - ...
    - ...
    - ...
    - 300 lines ...

```

With a role :

```

- hosts: all
  roles:
    - common
    - docker
    - nginx
    - app

```

This is much simpler and clearer. Especially since many validated and recognized roles are available on Ansible Galaxy!

7.2 Role Structure

A role is always organized as a file tree :

```

roles/
+-- my_role/
  +-- tasks/
    | +-- main.yml          # Main tasks
  +-- handlers/

```

```

|   +-- main.yml           # Handlers
+-- templates/
|   +-- config.conf.j2     # Templates
+-- files/
|   +-- file.txt           # Static files
+-- vars/
|   +-- main.yml           # Variables (high priority)
+-- defaults/
|   +-- main.yml           # Default variables (low priority)
+-- meta/
|   +-- main.yml           # Dependencies and metadata
+-- README.md

```

7.3 Creating a Role

```

# Initialize a role
ansible-galaxy init roles/nginx

# Structure created automatically:
# roles/nginx/
# +-- tasks/
# +-- handlers/
# +-- templates/
# +-- files/
# +-- vars/
# +-- defaults/
# +-- meta/

```

Listing 32 – Creating a role

7.4 Example : Nginx Role

defaults/main.yml :

```

---
nginx_port: 80
nginx_server_name: localhost
nginx_root: /var/www/html
nginx_enable_ssl: false

```

Listing 33 – Nginx role defaults

tasks/main.yml :

```

---
- name: Install Nginx
  apt:
    name: nginx
    state: present

- name: Create root directory
  file:
    path: "{{ nginx_root }}"
    state: directory
    owner: www-data
    group: www-data
    mode: '0755'

- name: Copy configuration template
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/sites-available/default

```



```

    owner: root
    group: root
    mode: '0644'
    notify: Reload Nginx

- name: Enable the site
  file:
    src: /etc/nginx/sites-available/default
    dest: /etc/nginx/sites-enabled/default
    state: link
    notify: Reload Nginx

- name: Start and enable Nginx
  service:
    name: nginx
    state: started
    enabled: yes

```

Listing 34 – Nginx role tasks

handlers/main.yml :

```

---
- name: Reload Nginx
  service:
    name: nginx
    state: reloaded

```

Listing 35 – Nginx role handlers

templates/nginx.conf.j2 :

```

server {
    listen {{ nginx_port }};
    server_name {{ nginx_server_name }};
    root {{ nginx_root }};
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}

```

Listing 36 – Nginx role template

7.5 Using a Role in a Playbook

```

---
- name: Deploy Nginx with a role
  hosts: webservers
  become: true
  roles:
    - nginx

  vars:
    nginx_port: 8080
    nginx_server_name: monsite.example.com

```

Listing 37 – Using a role in a playbook

7.6 Galaxy Roles

Ansible Galaxy is a repository of community-shared roles.

```
# Install a Galaxy role
ansible-galaxy install geerlingguy.nginx

# List installed roles
ansible-galaxy list

# Search for a role
ansible-galaxy search nginx
```

Listing 38 – Ansible Galaxy commands

7.7 Exercise : Refactor into Roles

Objective : Refactor the playbooks from previous parts into reusable roles.

Instructions :

1. Create an `nginx` role for Nginx deployment
2. Create an `ssh-hardening` role for SSH hardening
3. Create a `user-management` role for user management
4. Create a main playbook that uses these roles
5. Use default variables and override variables

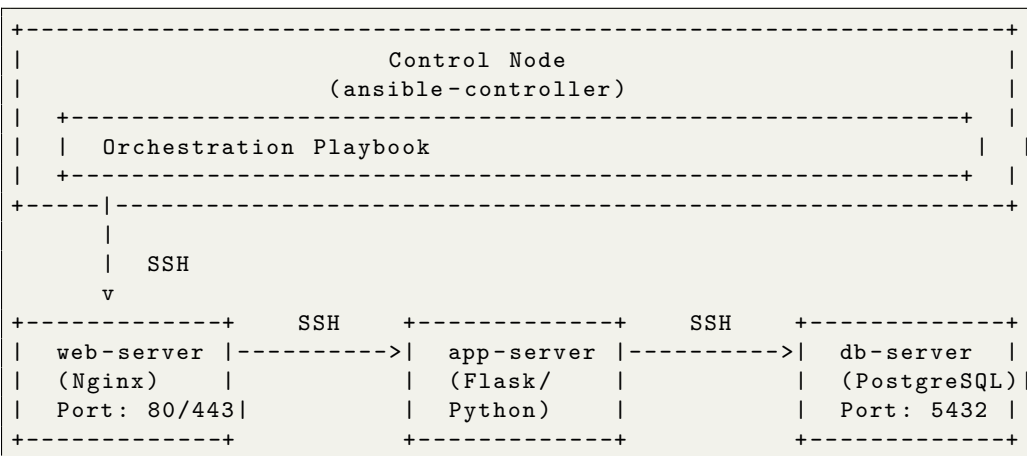
Validation criteria :

- Each role is independent and reusable
- Default variables are correctly defined
- The main playbook is simple and readable

8 Multi-Service Orchestration — Complete Architecture

8.1 3-Tier Architecture

In this part, you will deploy a complete 3-tier architecture :



Excalidraw diagram : `schemas/architecture-3tiers.excalidraw`

8.2 Multi-Host Inventory

```
# ~/ansible/inventory.ini

[webserver]
web-server ansible_host=<ip-web-server>

[appserver]
app-server ansible_host=<ip-app-server>

[dbserver]
db-server ansible_host=<ip-db-server>

[all:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_ed25519

[all:vars]
# Global variables
ntp_server="pool.ntp.org"
dns_server="8.8.8.8"

[webserver:vars]
# Variables specific to webserver
nginx_port=80
nginx_server_name=monsite.example.com

[dbserver:vars]
# Variables specific to dbserver
postgresql_version=15
postgresql_data_dir=/var/lib/postgresql/15/main
```

Listing 39 – Multi-host inventory

8.3 Orchestration Playbook

```
# ~/ansible/playbooks/06-orchestration.yml
---
- name: Database configuration
  hosts: dbserver
  become: true
  roles:
    - postgresql

- name: Application configuration
  hosts: appserver
  become: true
  roles:
    - python-app
  vars:
    db_host: "{{ hostvars['db-server']['ansible_default_ipv4']['address'] }}"
    db_name: myapp
    db_user: app_user
    db_password: "{{ vault_db_password }}"

- name: Web server configuration
  hosts: webserver
  become: true
  roles:
    - nginx
  vars:
    proxy_pass: "http://{{ hostvars['app-server']['ansible_default_ipv4']['address'] }}:5000"
```

8.4 Dependencies and Execution Order

Ansible executes plays in the order they are defined. To ensure services are deployed in the correct order :

```
---
- name: Step 1 - Database
  hosts: dbservers
  become: true
  tasks:
    - name: Install PostgreSQL
      apt:
        name: postgresql
        state: present

    - name: Wait for PostgreSQL to be ready
      wait_for:
        port: 5432
        timeout: 60

- name: Step 2 - Application
  hosts: appservers
  become: true
  tasks:
    - name: Wait for the database to be accessible
      wait_for:
        host: "{{ hostvars['db-server']['ansible_default_ipv4']['address'] }}"
        port: 5432
        timeout: 60

    - name: Install the application
      # ... installation tasks

- name: Step 3 - Web Server
  hosts: webservers
  become: true
  tasks:
    - name: Wait for the application to be accessible
      wait_for:
        host: "{{ hostvars['app-server']['ansible_default_ipv4']['address'] }}"
        port: 5000
        timeout: 60

    - name: Configure Nginx
      # ... configuration tasks
```

Listing 41 – Sequential deployment with wait_for

9 Advanced Security, Testing, and Best Practices

9.1 Ansible Lint

Ansible Lint is a linting tool that checks your playbooks against a set of rules.

```
# Install Ansible Lint
sudo apt install ansible-lint
```

```
# Lint a playbook
# Note: the first run may be VERY long (use the -vv option to see progress)
ansible-lint ~/ansible/playbooks/06-orchestration.yml

# Lint a role
ansible-lint roles/nginx/
```

Listing 42 – Ansible Lint commands

Common rules :

- `yaml` : invalid YAML syntax
- `command-instead-of-module` : use a module instead of `command`
- `no-changed-when` : tasks should return `changed_when` correctly
- `var-naming` : variables should follow a naming convention
- `security` : security issues (plaintext passwords, etc.)

9.2 Testing with Molecule

Molecule is a testing tool for Ansible that allows you to test your roles in isolated environments.

```
# Install Molecule
pip install molecule[docker]

# Initialize a test for a role
molecule init role my_role -d docker

# Run tests
molecule test
```

Listing 43 – Molecule commands

9.3 Security Best Practices

1. **Never store secrets in plaintext** : use Ansible Vault
2. **Use Ed25519 SSH keys** : more secure than RSA
3. **Apply the principle of least privilege** : don't use `become: true` everywhere
4. **Validate configurations** : use `validate` with modules that support it
5. **Log activities** : enable Ansible logging
6. **Version control playbooks** : use Git to track changes
7. **Test before deploying to production** : use Ansible Lint and Molecule
8. **Rotate keys and certificates** : automate rotation

Additional Resources

- **Official Ansible documentation** : <https://docs.ansible.com/>
- **Ansible Galaxy** : <https://galaxy.ansible.com/>
- **CIS Benchmarks** : <https://www.cisecurity.org/benchmark/linux>
- **Ansible Lint documentation** : <https://ansible-lint.readthedocs.io/>
- **Molecule documentation** : <https://molecule.readthedocs.io/>